

Distributed Shared Memory

Distributed shared memory(DSM) system is a resource management component of distributed operating system that implements shared memory model in distributed system which have no physically shared memory. The shared memory model provides a virtual address space which is shared by all nodes in a distributed system.

The central issues in implementing DSM are:

- how to keep track of location of remote data.

- how to overcome communication overheads and delays involved in execution of communication protocols in system for accessing remote data.

- how to make shared data concurrently accessible at several nodes to improve performance.

Architecture of Distributed Shared Memory (DSM) :

- Every node consists of 1 or additional CPU's and a memory unit. High-speed communication network is employed for connecting the nodes. A straightforward message passing system permits processes on completely different nodes to exchange one another.

Memory mapping manager unit :

- Memory mapping manager routine in every node maps the native memory onto the shared computer storage. For mapping operation, the shared memory house is divided into blocks.

Information caching may be a documented answer to deal with operation latency. DMA uses information caching to scale back network latency. the most memory of the individual nodes is employed to cache items of the shared memory house.



Memory mapping manager of every node reads its native memory as an enormous cache of the shared memory house for its associated processors. The basic unit of caching may be a memory block. Systems that support DSM, information moves between secondary memory and main memory also as between main memories of various nodes.

Communication Network Unit :

- Once method access information within the shared address house mapping manager maps the shared memory address to the physical memory. The mapped layer of code enforced either within the operating kernel or as a runtime routine.

Physical memory on every node holds pages of shared virtual-address house. Native pages are a unit of some node's memory. Remote pages in some other node's memory.



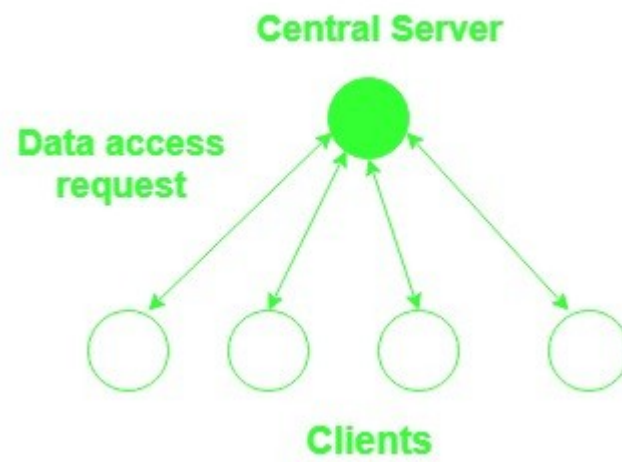
Algorithms to implement DSM

- 1. Central Server Algorithm:

In this, a central server maintains all shared data. It services read requests from other nodes by returning the data items to them and write requests by updating the data and returning acknowledgement messages.

- Time-out can be used in case of failed acknowledgement while sequence number can be used to avoid duplicate write requests.
- It is simpler to implement but the central server can become bottleneck and to overcome this shared data can be distributed among several servers. This distribution can be by address or by using a mapping function to locate the appropriate server.



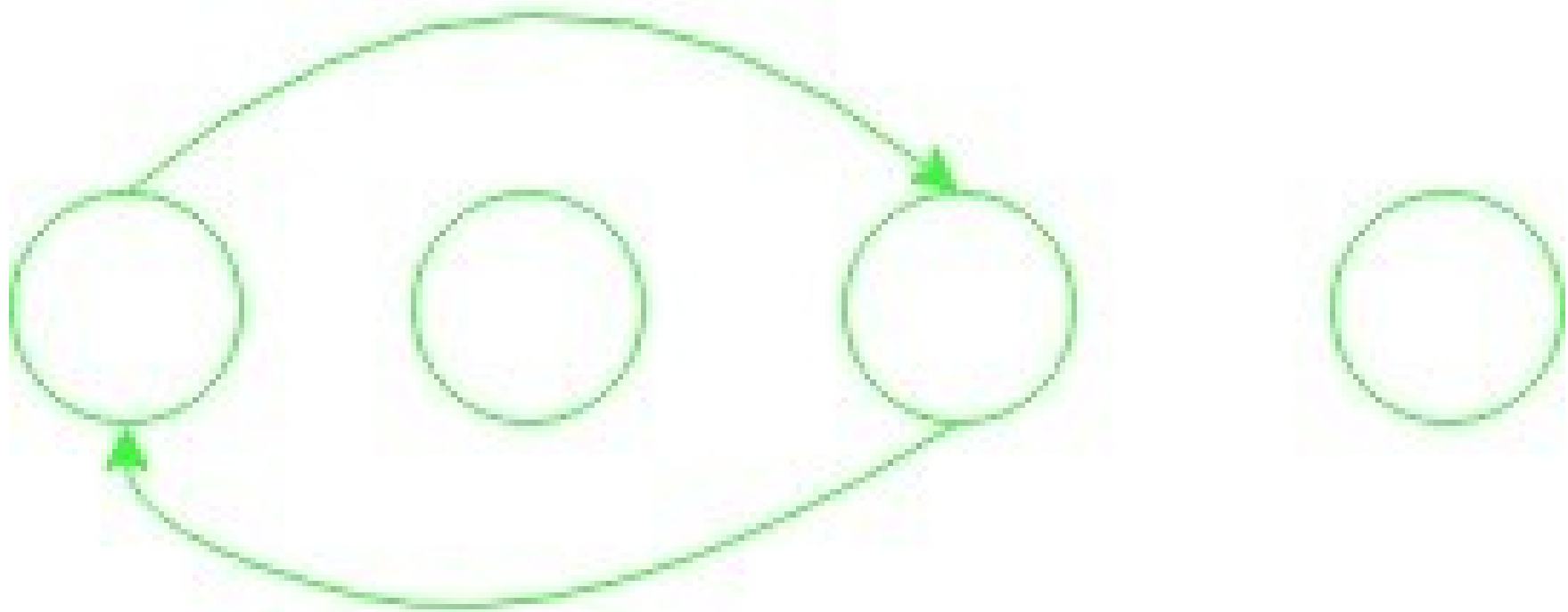


2. Migration Algorithm:

- In contrast to central server algorithm where every data access request is forwarded to location of data while in this data is shipped to location of data access request which allows subsequent access to be performed locally.
- It allows only one node to access a shared data at a time and the whole block containing data item migrates instead of individual item requested.
- It is susceptible to thrashing where pages frequently migrate between nodes while servicing only a few requests.
- This algorithm provides an opportunity to integrate DSM with virtual memory provided by operating system at individual nodes.



Data access request



Data block migrated

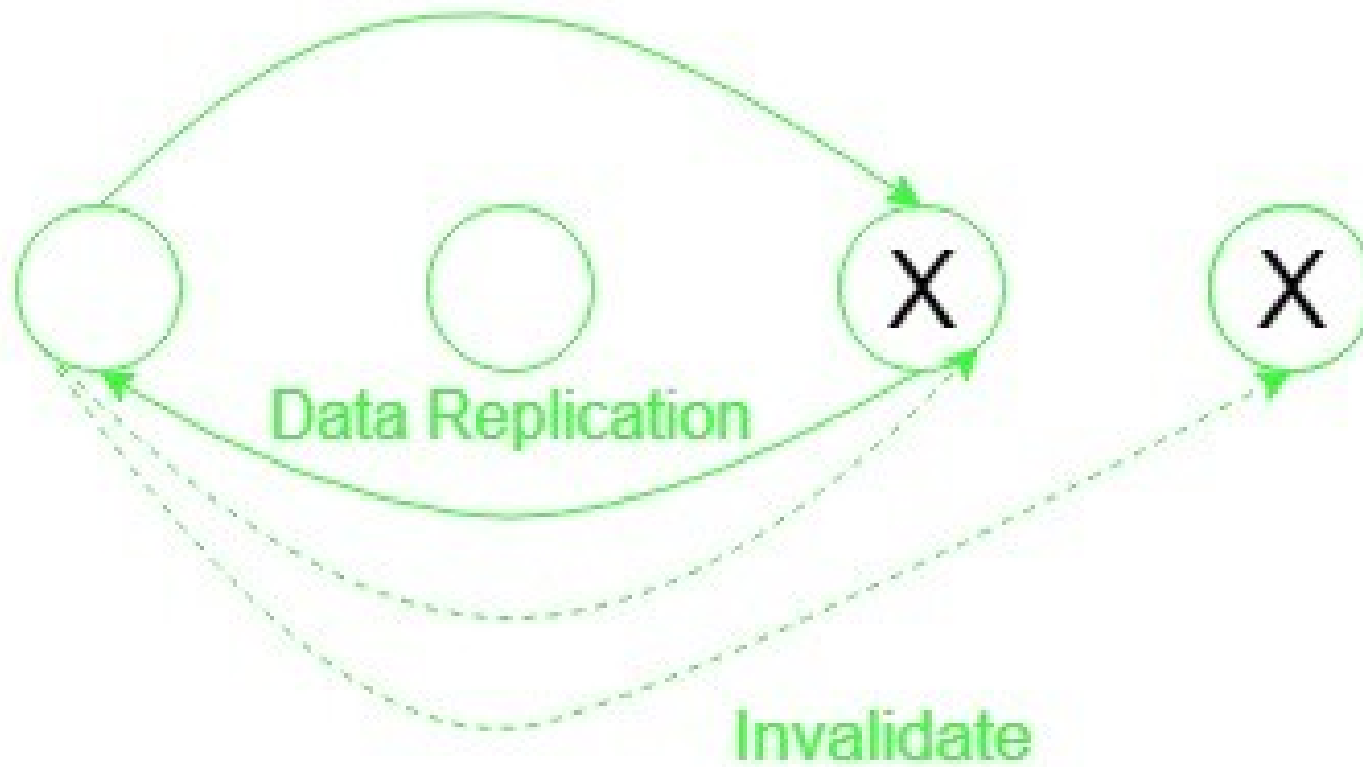


3. Read Replication Algorithm:

- This extends the migration algorithm by replicating data blocks and allowing multiple nodes to have read access or one node to have both read write access.
- It improves system performance by allowing multiple nodes to access data concurrently.
- The write operation in this is expensive as all copies of a shared block at various nodes will either have to be invalidated or updated with the current value to maintain consistency of shared data block.
- DSM must keep track of location of all copies of data blocks in this.



Data Access Request

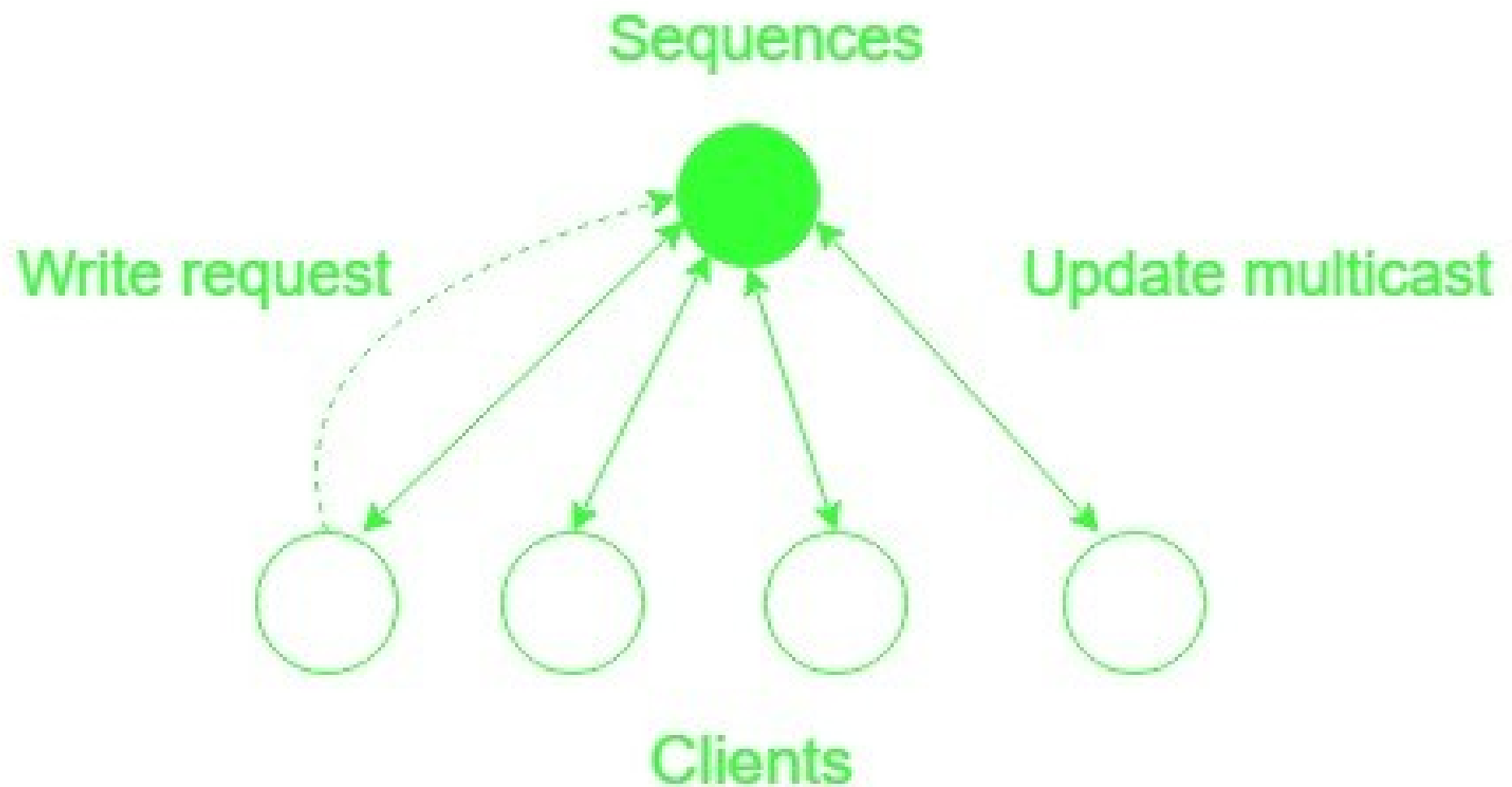


Write operation in Read Replication algorithm

4. Full Replication Algorithm:

- It is an extension of read replication algorithm which allows multiple nodes to have both read and write access to shared data blocks.
- Since many nodes can write shared data concurrently, the access to shared data must be controlled to maintain its consistency.
- To maintain consistency, it can use a gap free sequences in which all nodes wishing to modify shared data will send the modification to sequencer which will then assign a sequence number and multicast the modification with sequence number to all nodes that have a copy of shared data item.





Write operation in Full Replication algorithm

Issues in Load distributing

Introduction to distributed scheduling:-

- Scheduling refers to the execution of non-interactive processes or tasks at designated times and places around a network of computer
- Distributed scheduling refers to the chaining of different jobs into a coordinated workflow that spans several computers. For example: - you schedule a processing job on computer1 and computer2, and when these are finished you need to schedule a job on computer3, this is distributed scheduling.

If the task transfer involves large delay then using CPU queue length will make the load to accept more task while the already accepted task are in transit. When all the accepted task arrives at the node then the load becomes overloaded and requires further task transfer. This problem can be solved by artificially incrementing CPU queue length whenever a task is accepted. If the task transfer does not occur in a specified amount of time period then time out occurs and if time out occurs then CPU queue length is automatically decremented.

A decorative border at the bottom of the slide consisting of a grid of yellow hexagons with orange outlines. Some hexagons are filled with a solid orange color, creating a patterned effect.

LOAD:

Load on a system/node can correspond to the queue length of tasks/processes that need to be processed.

Queue length of waiting tasks: proportional to task response time, hence a good indicator of system loads.

Distributing load: transfer tasks/processes among nodes.

If a task transfer (from another node) takes a long time, the node may accept more tasks during the transfer time.

Causes the node to be highly loaded. Affects performance.

Solution: artificially increment the queue length when a task is accepted for transfer from remote node (to account for the proposed increased in load).

Task transfer can fail? : use timeouts.



Types of Load Distribution Algorithms:

Load Distribution Algorithms are classified as static, dynamic or adaptive. In static algorithms all load distribution decisions are hardwired in an algorithm on the basis of prior knowledge of the system. Dynamic algorithms make their load distribution decision on the basis of current system state, so they have more scope of improvement as compared to static algorithm. However the overhead incurred in correcting the system state information may outweigh the load benefit of load distribution.

Adaptive Algorithms are a special class of dynamic algorithms and they can adapt their activities by changing the parameter of the algorithm to suite the system state. Example, at high system load, dynamic algorithms continue the correction of system state information thereby further increasing the system load. On the other hand, adaptive algorithms will discontinue this procedure.



Load Balancing v/s Load Sharing :

Load Distribution Algorithms are further classified as load balancing and load sharing. The main aim of both the algorithms is to reduce the unshared state (a system in which some sites are idle while the other sites are heavily loaded) by transferring the task. Load Balancing algorithms goes one step ahead by equally distributing the load on all computers. Such algorithms have more task transfer so the overhead incurred in task transfer may overweight the potential performance improvement.

Preemptive v/s Non- Preemptive Transfer:

Preemptive tasks are those tasks which are partially executed. Their transfer is an expansive up gradation as the task state consisting of virtual memory image , process control block, list of higher buffers and timers is also to be transferred. Non-Preemptive tasks are those tasks which have not begin their execution such task transfer are also called task placement. In both type of task transfer, the environment (such as the privileges inherited by task) in which the terms are to be executed is also transferred.



Components of load distributing Algorithm

Distributed scheduling is concerned with distributing the load of a system among the available resources in a manner that improves overall system performance and maximises resource utilisation.

The basic idea is to transfer tasks from heavily loaded machines to idle or lightly loaded ones.

Load distributing algorithms can be classified as static, dynamic or adaptive. Static means decisions on assignment of processes to processors are hardwired into the algorithm, using a priori knowledge, perhaps gained from analysis of a graph model of the application. Dynamic algorithms gather system state information to make scheduling decisions and so can exploit under-utilised resources of the system at run time but at the expense of having to gather system information. An adaptive algorithm changes the parameters of its algorithm to adapt to system loading conditions. It may reduce the amount of information required for scheduling decisions when system load or communication is high.



A load distributing algorithm has, typically, four components:- transfer, selection, location and information policies.

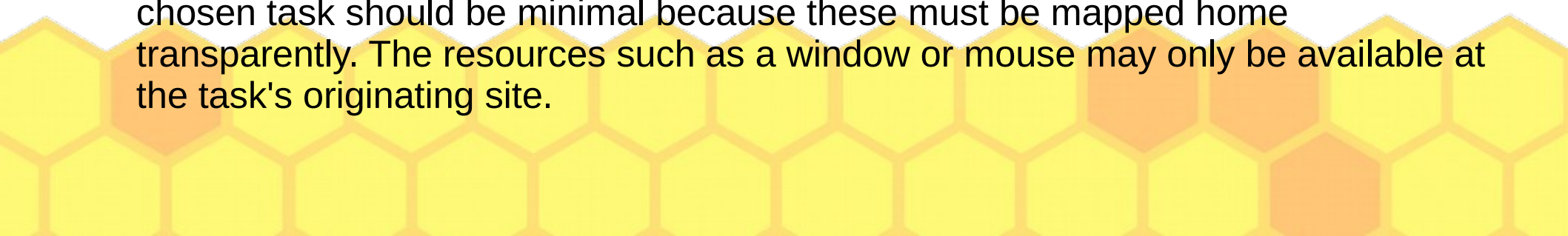
1. Transfer Policy

When a process is about to be created, it could run on the local machine or be started elsewhere. Bearing in mind that migration is expensive, a good initial choice of location for a process could eliminate the need for future system activity. Many policies operate by using a threshold. If the machine's load is below the threshold then it acts as a potential receiver for remote tasks. If the load is above the threshold, then it acts as a sender for new tasks. Local algorithms using thresholds are simple but may be far from optimal.

2. Process Selection Policy

A selection policy chooses a task for transfer. This decision will be based on the requirement that the overhead involved in the transfer will be compensated by an improvement in response time for the task and/or the system. Some means of knowing that the task is long-lived will be necessary to avoid needless migration. This could be based perhaps on past history.

A number of other factors could influence the decision. The size of the task's memory space is the main cost of migration. Small tasks are more suited. Also, for efficiency purposes, the number of location dependent calls made by the chosen task should be minimal because these must be mapped home transparently. The resources such as a window or mouse may only be available at the task's originating site.



3. Site Location Policy

Once the transfer policy has decided to send a particular task, the location policy must decide where the task is to be sent. This will be based on information gathered by the information policy.

Polling is a widely used sender-initiated technique. A site polls other sites serially or in parallel to determine if they are suitable sites for a transfer and/or if they are willing to accept a transfer. Nodes could be selected at random for polling, or chosen more selectively based on information gathered during previous polls. The number of sites polled may vary.

A receiver-initiated scheme depends on idle machines to announce their availability for work. The goal of the idle site is to find some work to do. An interesting idea is for it to offer to do work at a price, leaving the sender to make a cost/performance decision in relation to the task to be migrated.



Information Policy

The information policy decides what information about the states of other nodes should be collected and where it should be collected from. There are a number of approaches:

Demand Driven

A node collects the state of other nodes only when it wishes to become involved in either sending or receiving tasks, using sender initiated or receiver initiated polling schemes. Demand driven policies are inherently adaptive and dynamic as their actions depend on the system state.

Periodic

Nodes exchange information at fixed intervals. These policies do not adapt their activity to system state, but each site will have a substantial history over time of global resource usage to guide location algorithms. Note that the benefits of load distribution are minimal at high system loads and the periodic exchange of information therefore may be an unnecessary overhead.

State-Change Driven

Nodes disseminate state information whenever their state changes by a certain amount. This state information could be sent to a centralised load scheduling point or to peers.

Sender initiated Algorithms

Load balancing is a technique in which workload is distributed across multiple computers or other resources to get optimal resource utilization, minimum time delay, maximize throughput and avoid overload.

Load balancing scheme can be characterized as static and dynamic load balancing.

In dynamic load balancing scheme, tasks are transferred during execution from the heavily loaded processor to lightly loaded one.

In dynamic load balancing, at run time the changes are made to the distribution of work which uses the information about load that is current or recent for making distribution decisions.

A sender-initiated load balancing strategy is used by dynamic load balancing scheme.

A decorative pattern of yellow and orange hexagons at the bottom of the slide.

Basic Idea:

An overloaded node (sender) tries to send a task to an under loaded node (receiver).

Information policy:

Demand-driven: Upon identification of a node as a sender, load information from other nodes is collected. Among the possible load indexes, consider only CPU queue length.

Transfer policy: CPU queue threshold T for all nodes. Initiated when a task arrives.

Threshold based: A sender is a node, originating a task that makes its local CPU queue length exceed a threshold T . A receiver is a node that, by accepting a given task, would not exceed T .

Selection policy:

Newly arrived tasks or newly started processes without requiring preemption.

Location policy:

1. Random: No information exchange, Useless transfer are possible. Select any node to transfer the task at random. The selected node X may be overloaded. If transferred task is treated as new arrival then X may transfer the task again.
2. Threshold: Nodes are sequentially polled in random order. Poll nodes until a receiver is found which is done up to the Poll Limit nodes. If none is a receiver then the sender commits to the task.

3. Shortest: A random subset of nodes is polled and the one with the shortest queue length is chosen. Main aim is to choose the best receiver node rather than the first suitable one.

Stability of the algorithm:

In high load scenarios there won't (likely) be receivers, but polling would still keep on, triggered more and more frequently as task arrival rate increases.

The algorithm overhead can become higher and higher, eventually overcoming the service capacity of the system, causing system instability.



Receiver-initiated Algorithms

In receiver-initiated algorithms the load distribution activity is initiated from an under-loaded host (receiver), which tries to get a task from an overloaded host (sender)

a) Transfer policy: The threshold value for transfer policy depends on the present CPU queue length. The policy is triggered whenever a task departs. If the local queue length falls below the predefined threshold, then the host is identified as a receiver for obtaining a task from a overloaded host (sender). A host is identified to be a sender if its queue length exceeds the predefined threshold value T .

b) Selection policy: Unlike previous algorithms, receiver-initiated algorithms consider all tasks for transfer.

c) Location policy: The location policy selects a host at random and polls it to determine whether transferring a task would place its queue length below the threshold level. If not, then the polled host transfers a task.



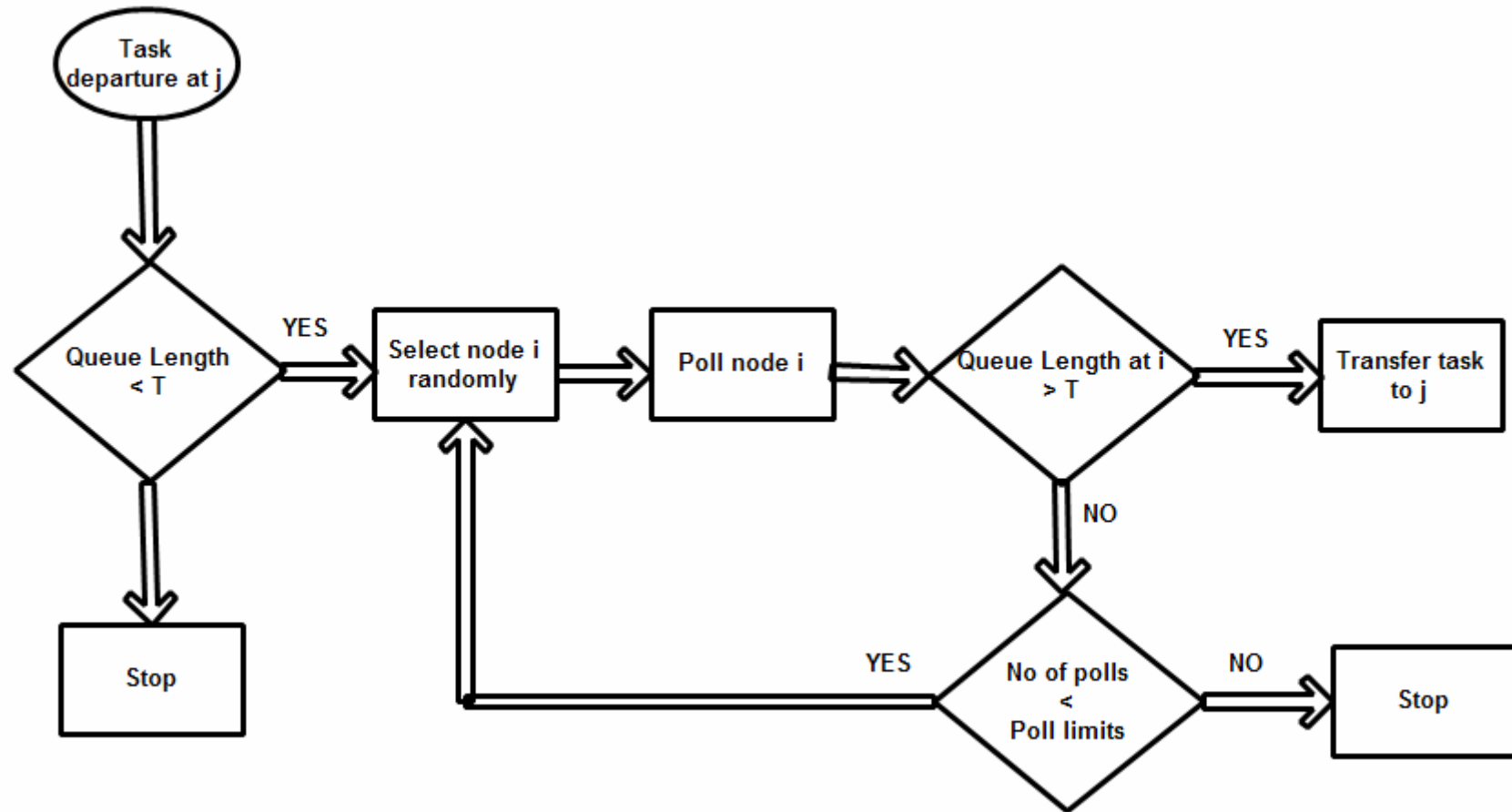
Otherwise, another host is selected at random, and the procedure is repeated until either a host that can transfer a task (a sender) is found or a static poll limit number of tries fail to find a sender. A problem with this location policy is that if all polls fail to find a sender, then the processing power available at a receiver is completely lost by the system until another task originates locally at the receiver (which may not happen for a long time). The problem severely affects performance in systems where only a few hosts generate most of the system workload and random polling by receivers can easily miss them.

Shivaratri et al suggested a remedy for this problem. If all the polls fail to find a sender, then the host waits until another task departs or for a predetermined period before reinitiating the load distribution activity, provided the host is still a receiver.

d) Information policy: The information policy is demand driven, since polling starts only after a host becomes a receiver.



- Receiver-initiated load sharing using a threshold location policy



Sender-initiated load sharing using a threshold location policy

